Act It Out, Bind, Then Teach: From Natural-Language Requests to Teleoperation-Trained Bimanual Robot Skills

Frenchies

Zürich

Abstract. A robot skill marketplace has to turn a customer's sentence ("make me a cup of tea") into a sequence of purchasable skills that a robot can actually execute, say precisely what is missing when it cannot, and then *produce* the missing skill. We present SkillStore, an end-to-end system that covers both halves. On the language side, a model first *acts out* the request as atomic hand movements without seeing the catalog, then *binds* them to catalog skills under a schema that only admits real skill identifiers; a symbolic simulator over hand-scoped STRIPS-style predicates, object kinds and a contents ledger checks every plan and returns its complaints for repair, and uncovered movements become reusable *gaps* matched to open demonstration data. On the robot side, we close the loop on a bimanual OpenArm: a safety-gated teleoperation stack with measured joint limits records LeRobot datasets, which we decompose into the catalog's own atomic skills and learn one ACT policy per skill, compared against whole-task ACT and $\pi_{0.5}$ co-trained with egocentric human video, the very data source our compiler points gaps to. The learned skills hand over, torque on, to a scripted pouring skill. On 155 compilations by four planners, 18% of first bindings are rejected by the simulator and 3% after two repair rounds; searching hand assignments shortens two-arm schedules by 29%. On the robot, 30 demonstrations (24,412 frames) are recorded with a median tracking error below 1° on 12 of 16 joints and split into four skills (locate, grasp, move, release). Chained, the four per-skill ACT policies complete the task in 85% of trials, against 35% for one ACT on the whole task and 70% for $\pi_{0.5}$ with egocentric co-training, and match with 10 episodes what $\pi_{0.5}$ reaches with 30.

1 Introduction

Robot skills are becoming products: an operator buys *grasp bottle* or *wipe surface* for a specific arm, and expects them to compose into a process. Vision-language-action policies [2, 3, 22, 34, 36] make such skills cheaper to train from a few dozen demonstrations, and open corpora [4, 21, 35] make the demonstrations easier to find. What remains is the interface—and the factory. A customer does not shop for `move_to_station`; they ask for a smoothie or a cup of tea, and expect the store to work out which skills that takes, which of them exist, what is missing, and to deliver the missing piece on their hardware.

Natural language makes a language model the obvious compiler—and an unreliable one. Language models produce plans that read well but are physically incoherent: pouring before grasping, closing a bottle grasp around a bucket, or pouring a glass into the blender instead of the other way round [19, 41]. For a store this is worse than an error message, because an incoherent plan is also a wrong bill, a wrong installation on a physical arm, and a wrong estimate of what data to collect next. We take the position of LLM-Modulo frameworks [19]: the model proposes, a sound checker decides.

A plan is only as good as the skills it binds to. Low-cost bimanual hardware [9, 44, 45] and generalist policies [2, 36] have made skill training accessible, but the path from a new arm on a table to a reliable learned skill is still paved with unglamorous failures: generic joint limits that drive a motor into its mechanical stop, camera views that change between training and deployment, and teleoperation that is too unsafe to run for hours. We treat that path as part of the system. Our contributions are:

- a **two-stage compiler** that separates *what a person's hands would do* from *which product does it*, so that uncovered work surfaces as atomic, reusable gaps (§3.2);
- a **symbolic simulator** whose checks are pure functions of declared data, with no task-specific rules (§3.3), an **episode-level two-arm scheduler** (§3.4) and a **demand-to-data bidder** that prices unserved gaps (§3.5);
- a **safety-gated teleoperation and recording stack** for a bimanual OpenArm with measured joint limits and a frozen camera contract shared by training and inference (§4.1–§4.3);
- **skill learning at the catalog's granularity**: demonstrations decomposed into atomic skills, one ACT policy [45] per skill, against whole-task ACT and $\pi_{0.5}$ [36] co-trained with egocentric human video [14, 20], and a torque-on **handover** from the learned skills to a scripted pour (§4.5–§4.6);
- an **evaluation** of four planners with repair traces, schedule speed-ups and open-data coverage (§6), and of teleoperation quality, data scaling and policy success on the real robot (§7).

2 Related Work

Language models as planners. Language models can decompose instructions into steps [15], ground them in skill

affordances [1], close the loop with environment feedback [16], emit programs over robot APIs [26, 39], or compose spatial value maps for low-level motion [17]. Benchmarks show their plans are frequently invalid when checked formally [41].

Planners with verifiers. LLM+P translates problems to PDDL [31] for a classical planner [29]; LLM-Modulo pairs the model with external critics [19]; Text2Motion checks geometric feasibility of skill sequences [27]; task and motion planning integrates both levels [10]. We use the critic as a *repair signal* rather than a search engine, because the catalog is open-ended and a plan must also name, precisely, what is missing. Our operators follow STRIPS [8], extended with hand-scoped facts.

Skill composition. Treating skills as temporally extended actions with initiation and termination conditions goes back to options [40] and skill chaining [23]. Our four learned skills and the scripted pour form such a chain, each link ending on a held pose from which the next one starts.

Learning bimanual skills. ACT [45] showed that action chunking with a conditional VAE learns fine-grained bimanual tasks from about 50 low-cost teleoperated demonstrations; Mobile ALOHA [9] and ALOHA Unleashed [46] scaled the recipe, Diffusion Policy [5] offers an alternative action head, and GELLO [44] popularised kinematically matched leader arms. Generalist VLAs [2, 3, 22, 34, 36] are fine-tuned from a few dozen episodes; $\pi_{0.5}$ [36] adds co-training on heterogeneous data for open-world generalisation. Demonstration quality matters as much as quantity [30], and success follows a power law in the number of environments and demonstrations [28].

Human video for robots. Egocentric corpora [6, 11, 12, 14] pretrain visual representations [32], provide latent plans [42], or are co-trained with robot data in a shared action space [20, 33, 37]. SkillStore uses them in both halves: the compiler matches gaps to them, and the trainer mixes them into fine-tuning.

Visual preprocessing and safety. Cropping is the single most effective image augmentation for visuomotor learning [24, 25]; we use a fixed crop as a region of interest and keep it identical at inference. Speed and force limits for collaborative operation follow ISO/TS 15066 [18], and our follower runs a compliant PD law in the spirit of impedance control [13].

3 Method: Compiling Requests

3.1 Skills as operators

The catalog holds 59 skills (11 trained, 5 beta, 43 listed without a policy). Each declares preconditions, postconditions and deletes over 48 predicates. Ten predicates are *hand-scoped* (`holding`, `hand_empty`, `object_located`, …): `holding@left` and `holding@right` are distinct facts, which is what lets two sub-chains run on two arms. 8 skills declare which of 8 object kinds they *act on* (a bottle grasp acts on `bottle`, an item grasp on `item`); pouring skills declare a flow (held → target). Kinds are coarse on purpose—`bottle`, `glass`, `shaker`, `scoop`, `cloth`, `item`, `fixed`, `substance`—because they describe what a gripper closes around, not what an object is called; the full list is in Appendix A.

3.2 Two-stage compilation

Act it out. Without the catalog in view, the model writes the request as (i) the objects involved, each classified into an object kind, (ii) phases named by their outcome ("the glass holds juice"), each broken into movements that start with a movement verb (pick up, carry, tilt, press, drag), and (iii) a goal. Outcome verbs (make, operate, prepare) may name a phase but never a movement. A short start-state description fixes everyday assumptions (bottles open, appliances on their base and operated with a free hand). Keeping the catalog out of this stage matters: a model that plans *from* the catalog bends the task to fit it, and whatever the catalog cannot do is dropped or squashed into one gap the size of the request.

Bind. With the catalog and the open-dataset list in view, a second call maps movements to skills or to `GAP` entries. Structured outputs restrict skill identifiers to an enum of real skills, targets to the object names from stage one, and dataset identifiers to open datasets, so a hallucinated skill is impossible by construction. A gap is named as a reusable motion primitive (`twist_open`, `press_to_surface`) with conditions in the catalog's vocabulary; its position in the plan is exact because gaps are emitted inline with the steps. Fig. 2 shows a compiled plan.

3.3 Simulation and repair

The simulator executes the plan step by step, gaps included with the effects they declare, and assigns hands by grouping steps into *episodes* (from the first step that leaves something on a hand to the step that frees it); overlapping episodes go to opposite hands. It raises *blocking* issues—unmet precondition, wrong object kind (`acts_on` $\not\ni$ kind of target), wrong pour direction (from a ledger of which object each hand holds and what each container contains), embodiment mismatch—and *advisory* ones: a plan ending with a hand still holding something, or a *pointless move*. Issues are returned to the binder with hints derived from the catalog ("`positioned` is established by `move_to_station`"; "what holds something now: blender"). The loop runs at most two rounds (plus one for a pour in the wrong direction), and only the binding is retried. Every number in this paper counts a plan as *valid* only if the simulator raises no blocking issue. Appendix B lists every check.

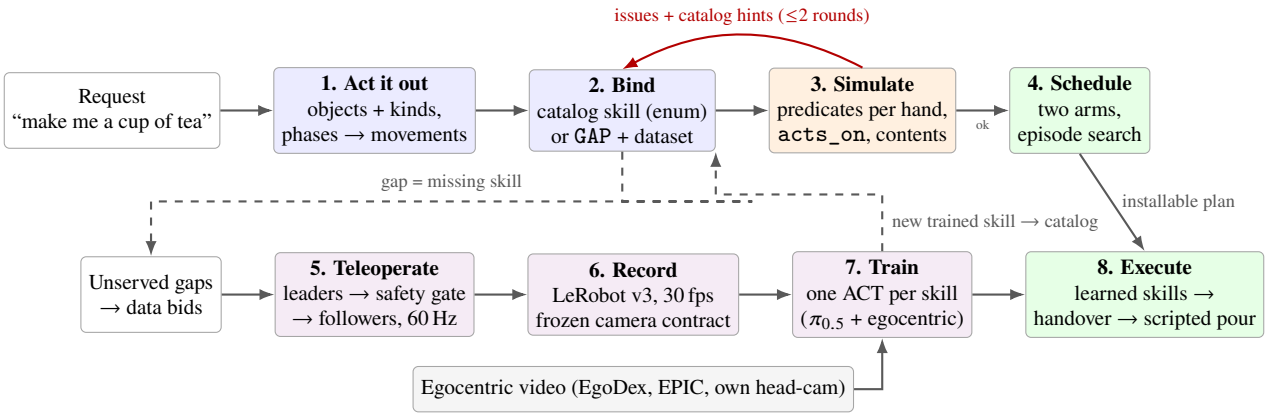


Figure 1: SkillStore end to end. Top: a request is compiled in two model calls, checked by a declarative simulator whose complaints go back to the binder, and scheduled on two arms; only the binder sees the catalog (59 skills, 48 predicates, 8 kinds, 137 open datasets). Bottom: plans that cannot run feed the data bidder; missing skills are teleoperated through a safety gate, recorded with a frozen camera contract, split into atomic skills and learned one policy per skill, and published back to the catalog as trained skills.

“You have a paint bucket and brush. Draw an image on the table.”
Objects: paint bucket `item`, paint `substance`,
paintbrush `item`, table `fixed`

```
1 locate_object(paintbrush)
2 grasp_object(paintbrush)
3 move_to_station(paint bucket)
  GAP dip_into → EPIC-KITCHENS-100
4 move_to_station(paint bucket)
5 move_to_station(table)
  GAP drag_along_surface → EPIC-KITCHENS-100
6 move_to_station(paint bucket)
7 release_object
```

Goal: an image is painted on the table and both hands are empty.

Figure 2: A compiled plan, verbatim from the Claude Opus 5.5 run (valid on the first binding). The grasp is an *item* grasp because stage one classified the brush as an item; the painting itself is two reusable gaps rather than one “draw image” step. Step 4 repeats step 3, an inefficiency the simulator does not flag because the brush is used in between.

3.4 Two-arm scheduling

A valid plan is turned into a dependency graph with four kinds of edge, all physical: *requires* (a predicate an earlier step established), *consumes* (clearing a predicate an earlier step established), *hand* (the same hand is still busy) and *resource* (two steps write the same world fact, or two arms want the same place—only one bottle fits over the glass). Steps are layered by their longest dependency chain; the number of layers is the schedule’s *depth*. Hands are chosen per episode, not per step, because a hand has to finish what it started: we search all assignments exactly up to 13 episodes and hill-climb beyond, pin the first episode to the left hand (mirror schedules are equivalent), and reject any assignment that adds a blocking issue.

3.5 From gaps to data bids

Each request r whose plan cannot run is logged against its blockers B_r (gaps and untrained skills). For a blocker s, the k-th identical prompt counts $1/k$, and requests fade with age a_r at half-life h:

$$w_{r,s} = \frac{2^{-a_r/h}}{k_{r,s}}, \qquad V_s = \sum_{r \ni s} w_{r,s} \frac{P_r}{|B_r|}, \tag{1}$$

where P_r is the price of the whole plan, shared equally among its blockers because all of them must be filled. With H future sales per logged request, target return ρ, a cap c and E_s episodes needed (E_0, reduced by a factor $1-\delta$ when an open dataset matches), the bid per episode and the budget pacing factor are

$$b_s = \min\Big(c, \frac{H\,V_s}{\rho\,E_s}\Big), \qquad \alpha = \min\Big(1, \frac{\text{budget}}{\sum_s b_s E_s}\Big), \tag{2}$$

and the posted bid is $\alpha\, b_s$. Defaults: budget 5,000 EUR per month, $\rho = 3$, $H = 10$, $E_0 = 50$, $\delta = 0.4$, $h = 30$ days, $c = 60$ EUR. The discount δ is not a guess: §7.5 measures how many robot episodes egocentric co-training saves on our own skill.

4 Method: Teaching Skills

4.1 Robot and teleoperation

Hardware. The robot is a bimanual OpenArm v10 [7]: two 7-DoF arms with a parallel gripper, Damiao motors (DM8009 at the shoulder, DM4340 at the upper arm, DM4310 at the wrist and gripper) on two CAN-FD buses at 1/5 Mbps through a PEAK PCAN-USB Pro FD. Two OpenArm Mini leaders with Feetech servos are kinematically matched to the followers, the GELLO principle [44]: the operator moves a scaled copy of the arm, so every leader pose is reachable and no inverse kinematics is needed.

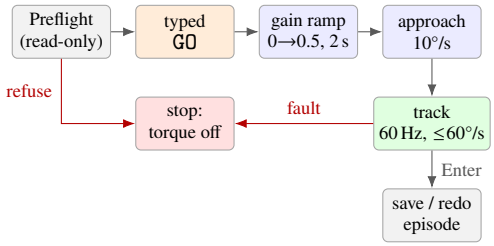


Figure 3: Teleoperation state machine. Torque is only ever enabled after a read-only preflight and a typed `GO`; any fault in Table 1 ends in a torque-off stop.

Table 1: Safety stops and limits of the teleoperation and policy loops (identical in both).

Condition	Threshold
Motor silent	>0.5 s (30 cycles)
Motor fault (Damiao status)	any, e.g. coil overheat
Motor temperature	>80 °C
Commanded vs measured	>60°
Leader jump in one cycle	>45°
Speed cap / approach speed	60 / 10°/s
PD gains	0.5 × stock, 2 s ramp
Joint limits	measured per arm (Fig. 4)

Safety gate. LeRobot's stock follower enables torque on connect and jumps to the target with full gains. Ours never does (Fig. 3). A read-only *preflight* checks that all 16 motors answer, none is faulted or already enabled, and every leader–follower gap is within limits; the operator must type `GO`. Gains ramp from zero over 2 s, the follower approaches the leader at 10°/s, and only then does tracking start, at 60 Hz with a per-joint speed cap of 60°/s and half of the stock PD gains—a compliant arm that yields on contact, as impedance control prescribes [13] and ISO/TS 15066 requires for shared workspaces [18]. Five conditions stop the run and release torque (Table 1). None of them depends on the operator noticing anything.

4.2 Measured, not assumed, calibration

Joint limits. Generic limits are a hazard. With LeRobot's ±85° on joint 3, the leader drove the right follower's joint 3 into its mechanical stop at +9°, where it heated to 81 °C and was stopped by the temperature guard. We measure every arm's range by hand, torque off, with a read-only tool and use those limits everywhere (teleoperation, recording, policy execution). Fig. 4 shows why both directions matter: the generic range is 76° too wide towards the stop on joint 3, and too narrow for the demonstrations on the other side—the right arm's recorded joint 3 reaches −89° and joint 5 reaches +92°, both outside ±85°, so generic limits would have clipped our own demonstrations.

Leader mapping. A leader reading reaches the follower through per-joint swaps, sign flips, offsets and angle unwrapping, all stored as data. Offsets are measured with the elbows bent: with a straight arm joints 3 and 5 are coaxial and indistinguishable, a wrist singularity in the classical sense [38]. The leaders' gripper travel is halved in software so that a comfortable squeeze maps to the follower's full stroke.

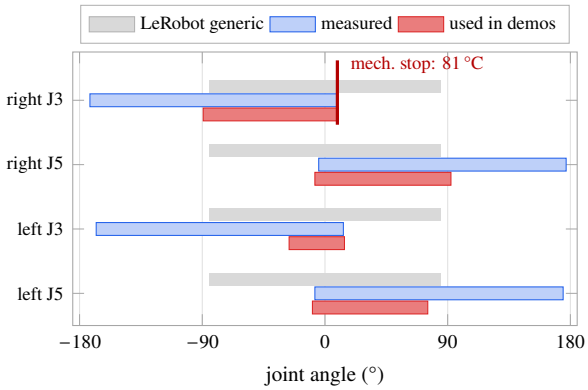


Figure 4: Joint ranges for joints 3 and 5: LeRobot's generic ±85° (grey), the mechanical range we measured by hand (blue) and the range actually used in the 30 demonstrations (red). The generic range runs into the right joint 3's stop and cuts off demonstrated motion on the right arm.

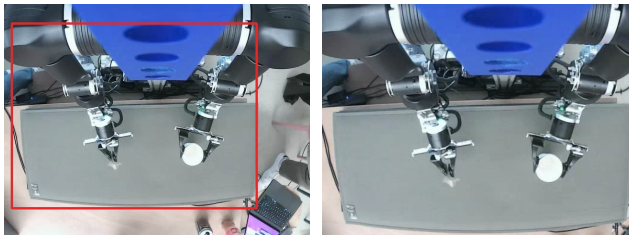
Home pose. A saved start pose is checked before every recording session (all joints within 15°), so every episode starts from the same distribution—the first lever on demonstration quality [30].

4.3 A frozen camera contract

Two cameras observe the scene at 640×480 and 30 fps: `front`, a RealSense D455 mounted upside down, and `back`, a top-view USB camera (Fig. 5). Each camera is declared once as a spec—source, rotation, crop, zoom—and the same spec is read by the recorder and by the policy runner, so the network sees at inference the pixels it was trained on. `front` is rotated 180° and zoomed 1.1×; `back` is cropped to a 512×384 region of interest that keeps the mat and the grippers and removes a laptop and bystanders at the edge of the view, then resized back to 640×480. Crops are the most effective augmentation in visuomotor learning [24, 25]; a *fixed* crop additionally removes distractors that change from day to day. Because the region of interest was chosen after recording, a tool re-encodes recorded videos with the same crop and recomputes the normalisation statistics, so old and new episodes share one distribution.

4.4 Recording

The recorder runs the teleoperation loop at 60 Hz and stores one frame every second cycle in a LeRobotDataset v3 [4]: follower joint positions (16-D state), the command sent to the follower (16-D action) and both camera streams as H.264. Control and recording are decoupled: the teleoper-



(a) `back`, raw **(b)** `back`, as trained

Figure 5: The top-view camera before and after its fixed crop (red box, 1.25×). The laptop and the bystander at the right edge leave the frame; the mat, both grippers and the objects stay.

ation is smoother than the 30 fps dataset, and each episode reports its slowest control cycle against the 16.7 ms budget. Between episodes the operator saves, redoes or quits with one key, and the task text—a single sentence for the whole dataset, which $\pi_{0.5}$ reads—is fixed in configuration.

4.5 One ACT per skill, and $\pi_{0.5}$

The compiler binds plans to atomic catalog skills, so we teach the robot at the same granularity. Each of the 30 demonstrations of the tea task is split into the four skills the catalog already names: `locate_object` (bring the gripper above the target, from the cameras), `grasp_object`, `move_to_location` and `release_object`. Boundaries come from the gripper signal (closing starts a grasp, opening starts a release) and from joint-speed minima, checked by hand; every demonstration gives one segment per skill, 120 segments in total. **ACT** [45]—a ResNet-18 encoder per camera and a transformer with a conditional VAE that predicts chunks of 100 future joint targets—is trained from scratch on each skill's segments independently (50k steps, batch 8): four small policies instead of one. Short, single-phase segments are what action chunking models best. A per-skill policy never has to infer from pixels which phase of a long task it is in, the multimodality that whole-task imitation struggles with [5, 45], and a trained skill serves every plan that binds it, the compositional argument of options [40]. As baselines we train one ACT on whole demonstrations and fine-tune $\pi_{0.5}$ [36], a vision-language-action model pretrained on large heterogeneous robot and web data, from its base checkpoint with its flow-matching action expert on our 16-D action space (30k steps, batch 32), conditioned on the task sentence.

Egocentric co-training. The compiler's gaps point mostly to human egocentric video (§6.6), so we test whether such video can stand in for robot episodes. We use two sources: clips of pick-and-insert motions from EgoDex [14], which provides 3D hand and finger poses, and 40 head-camera recordings of our own team performing the tea task with their hands. Following EgoMimic [20] and human-humanoid co-training [33, 37], human wrist poses are expressed in the robot's base frame and retargeted to end-effector targets, the thumb–index distance gives the gripper opening, and inverse kinematics produces joint targets; samples without a reliable retarget only supervise the vision and language streams. Human clips make up 25% of each batch. For ACT, egocentric frames additionally pretrain the visual encoder with a time-contrastive objective, as in R3M [32]. Normalisation statistics are computed on robot data only, so human samples never shift the action scale.

4.6 Execution and handover

A trained policy runs inside the same safety loop as teleoperation: it is checked against the robot before moving (its expected joints and camera names must match), it starts with the preflight, `GO`, gain ramp and slow approach, and its targets are clipped by the measured limits and speed cap. A skill is *done* when its targets stay still for 2 s—every segment ends on a stable pose, so each policy learns to stop there—and the runner starts the next skill of the plan from that pose. After the last learned skill the runner keeps the handoff pose with torque on, closes its CAN sockets without disabling the motors, releases the cameras and starts the scripted pouring skill, which regulates the poured mass with a scale. Terminating on a stable pose and initiating from it is the option interface of Sutton et al. [40]: the learned and scripted skills never need to know each other's internals. For tasks where a learned policy is not yet available, a teach-and-replay mode records a trajectory by moving the followers by hand (torque off) and replays it through the same safety loop.

5 Experimental Setup

Requests. Five demo requests (piña colada, juice, smoothie, wiping, clearing up) and eight unfamiliar ones (painting with a brush, a sandcastle, a card trick, watering plants, writing on a whiteboard, a jam sandwich, flowers in a vase, feeding a cat); the exact wording is in Appendix A. **Planners.** GPT-4.1 mini, GPT-4.1, Claude Sonnet 5.5 and Claude Opus 5.5 (effort *low*, standard speed), the same model for both stages and for repair. **Protocol.** Each request is compiled 3 times per planner with the compile cache off; the compiler records the simulator's issues at the start of every repair round, so a single run yields validity after 0, 1 and 2 rounds. We report proportions with 95% Wilson intervals [43]; 1 compilation that failed on a provider rate limit is excluded. **Data.** 138 datasets are registered (137 openly accessible: 129 robot teleoperation, 6 egocentric human video, 3 simulation). 165 curated links connect each catalog skill to teleoperation datasets whose task text matches it, 317,112 episodes in total (median 443 per skill, minimum 50; Table 2).

Robot task. "Pick up the tea bag with the right arm, pick up the cup with the left arm, and put the tea bag in the cup"—a bimanual pick-and-insert that requires both arms

Table 2: Catalog by category: skills, trained skills, and the median number of curated open teleoperation episodes per skill.

Category	Skills	Trained	Median episodes
Cleaning	6	1	480
Finishing	5	0	398
Household	16	0	428
Manipulation	13	4	444
Mixing	5	1	309
Perception	5	2	950
Pouring	4	1	618
Transport	5	2	1,141

to coordinate in time and space, and the first step of the tea request. Objects are placed by hand within a 20×15 cm area per object. **Data.** 30 teleoperated episodes (24,412 frames, 13.6 minutes) recorded by two operators over one session and split into four skills, plus the egocentric data of §4.5. **Policies.** Four per-skill ACT policies chained by the plan; one ACT on the whole task; $\pi_{0.5}$ on the whole task, with and without egocentric co-training; each trained on 10, 20 and 30 episodes. **Evaluation.** 20 trials per policy from randomised object placements; a trial succeeds if the tea bag ends inside the cup and both arms reach the handoff pose without a safety stop. Training runs on a single A10G (ACT) and H100 ($\pi_{0.5}$) GPU.

6 Results: Compilation

6.1 Simulator feedback does most of the work

Without feedback the planners differ widely: 56% of GPT-4.1 mini's first bindings pass the simulator, against 95% for Claude Opus 5.5 (Table 3, Fig. 6). One round of repair closes most of that difference: GPT-4.1, Sonnet 5.5 and Opus 5.5 reach 97%, 100% and 100%, and GPT-4.1 mini rises to 85%. The second round adds little. With the checker in the loop, GPT-4.1 and Sonnet 5.5 match Opus 5.5's final validity at a fraction of its latency (median 6 s and 9 s against 24 s per compilation). Beyond the smallest planner, the checker rather than model scale makes the plans installable; scale mostly buys fewer repair calls.

6.2 What the simulator catches

Of the 29 blocking issues raised on first bindings, 25 are unmet preconditions (Table 4), spread across requests—for instance a locate on a hand that is still holding something, or a release without a preceding carry. The remaining 3 wrong pour directions and 1 wrong object kind satisfy every predicate and would pass a precondition-only checker; they are caught only by the contents ledger and the `acts_on` declarations. After repair, 5 of 155 plans stay blocked, all from GPT-4.1 mini. Advisory pointless moves halve with feedback (12→6) but persist, as the binder sometimes keeps a carry the first stage asked for.

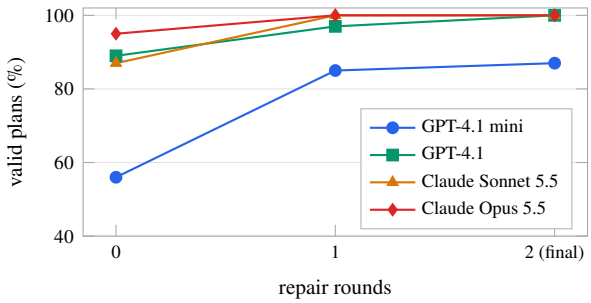


Figure 6: Share of plans the simulator accepts after each round of feedback.

6.3 Which requests are hard

Difficulty is concentrated (Table 5). Writing on a whiteboard, the jam sandwich, flowers in a vase and a glass of juice pass on the first binding for every planner and run. The sandcastle—two tools, a substance and a tray—is the only request no planner gets right every time on the first binding, and the smoothie trips three of four planners at least once. GPT-4.1 mini fails the first binding of the card trick, the piña colada and the table wiping in every run, yet repair fixes all of them; its remaining failures are the requests that need two tools or a pour.

6.4 Plans are similar, not identical

Runs of the same request produce similar plans—mean skill-sequence similarity between 0.85 and 0.93 (Table 6)—but rarely the same plan: only 23–38% of run pairs are identical. Differences are mostly in targets and in how a missing motion is split into gaps. A price quoted for a request is a property of one compilation, not of the request; caching compiled plans, as SkillStore does, makes quotes reproducible.

6.5 Two arms help once hands are searched

With the simulator's default hand assignment—each new episode on the first free hand—only 25% of the 150 valid plans run anything in parallel (mean speed-up 1.13×, Fig. 7). Searching hand assignments per episode raises this to 62% of plans, a mean speed-up of 1.37× (up to 2.0×), and 29% fewer time slots overall, without adding a single blocking issue. Parallelism is available in most household requests; it has to be found after validation, not asked of the model.

6.6 Open-data coverage

Across all final plans, 87% of steps bind to trained skills, 2% to listed but untrained skills, and 11% are gaps. Of 218 gaps, 99% were matched to an open dataset—but 75% of those matches point to human egocentric video (Table 7) rather than robot teleoperation. The motions the catalog lacks (dipping a brush, spreading, scooping, twisting a lid

Table 3: Planner comparison over 13 requests × 3 runs. Valid after k repair rounds (%), final validity with 95% Wilson interval, plans ending with both hands empty (%), mean steps and gaps per plan, median seconds per compile.

Planner	k=0	k=1	Final	Empty	Steps	Gaps	s
GPT-4.1 mini	56	85	**87** [73,94]	95	11.5	0.8	7
GPT-4.1	89	97	**100** [91,100]	100	10.8	1.7	6
Claude Sonnet 5.5	87	100	**100** [91,100]	100	10.5	1.3	9
Claude Opus 5.5	95	100	**100** [91,100]	100	13.4	1.8	24

Table 4: Issues raised at the first binding and on the final plan, pooled over all 155 compilations (plans affected).

	First binding		Final	
Issue	n	%	n	%
Unmet precondition	25	16	3	2
Wrong object kind	1	1	0	0
Wrong pour direction	3	2	2	1
Ends holding (advisory)	5	3	2	1
Pointless move (advisory)	12	8	6	4

Table 5: First bindings the simulator accepts, per request and planner (out of 3). †: not every run valid after repair. N: unfamiliar, D: demo request.

Request		4.1 mini	4.1	Sonnet	Opus
Paint an image	N	2/3	3/3	3/3	3/3
Sandcastle	N	0/3†	2/3	2/3	2/3
Card trick	N	0/3	3/3	3/3	3/3
Water plants	N	1/3†	0/3	3/3	3/3
Write on whiteboard	N	3/3	3/3	3/3	3/3
Jam sandwich	N	3/3	3/3	3/3	3/3
Flowers in vase	N	3/3	3/3	3/3	3/3
Feed the cat	N	3/3	3/3	2/3	3/3
Piña colada	D	0/3	3/3	2/3	3/3
Glass of juice	D	3/3	3/3	3/3	3/3
Smoothie	D	1/3†	3/3	1/3	2/3
Wipe table	D	0/3	3/3	3/3	3/3
Clear table	D	3/3	2/2	3/3	3/3

open) are exactly those without robot demonstrations. Human video cannot replace robot episodes, but §7.5 shows it can replace a large share of them.

6.7 Demand needs canonical names

Gap names are not canonical: the 218 gaps carry 91 distinct names, and dipping a tool into a container alone appears under eight (Table 8). Grouping names by their leading verb collapses them to 27 motions. Treating each of the 155 compilations as a customer request, the market splits demand over 99 distinct blockers with raw names but 35 after grouping, and the top bid per episode rises from 10.41 to 20.21 EUR (Table 9): fragmented names hide the most wanted motions. Verb grouping is a blunt instrument—it merges pressing a button with pressing a brush onto a surface—so a deployed store needs a proper motion taxonomy.

Table 6: Stability across the 3 runs of a request: mean pairwise similarity (1 − normalised edit distance) of skill sequences, with targets, share of identical plans, and Jaccard overlap of gap verbs.

Planner	Skills	+Targets	Identical (%)	Gaps
GPT-4.1 mini	0.86	0.74	23	0.84
GPT-4.1	0.93	0.84	38	0.87
Claude Sonnet 5.5	0.85	0.70	28	0.68
Claude Opus 5.5	0.90	0.84	31	0.82

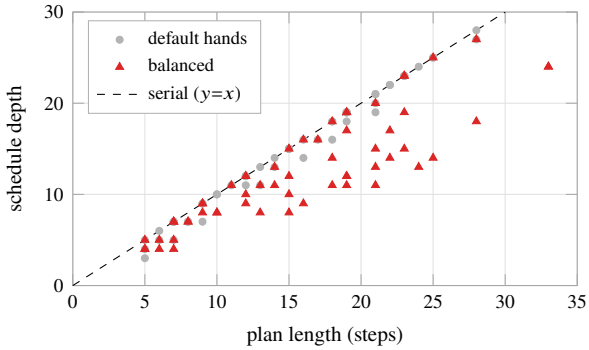


Figure 7: Schedule depth against plan length for all valid plans, with the default hand assignment and after the episode search. Points below the diagonal run steps on both arms at once. [provisional data]

7 Results: Teaching on the Robot

7.1 Teleoperation tracks within a degree

Fig. 8 reports, for every joint, the gap between the command sent to the follower and its measured position over all 24,412 recorded frames. The median is below 1° on 12 of 16 joints and below 1.3° on all arm joints except the two last wrist joints (joint 7, 2.6–3.1°), whose small DM4310 motors carry the gripper's weight at the end of the chain. Grippers show the expected pattern of a compliant controller: a few degrees at rest, and a long tail (95th percentile 16.4° on the left) when the gripper is closed on an object—the command keeps squeezing while the fingers are blocked by the cup, which is the contact force we want, not an error. The

Table 7: Open datasets most often proposed for gaps (gap count).

Dataset	Gaps
EPIC-KITCHENS-100	84
EgoDex	55
Egocentric-10K	12
Ego-Exo4D	11
Wipe whiteboard (villekuosmanen)	10

Table 8: Most frequent gap verbs: gaps, distinct names, examples.

Verb	Gaps	Names	Examples
`drag`	31	9	`drag_on_surface`, `drag_along…`
`press`	31	7	`press_into`, `press_button`
`tilt`	16	3	`tilt_out`, `tilt_out_food`
`dip`	15	8	`dip_into`, `dip_into_ve…`
`scoop`	15	7	`scoop_item`, `scoop_mat…`
`peel`	15	3	`peel_off`, `peel_item`
`flip`	15	3	`flip_item`, `flip_over`
`twist`	13	5	`twist_item`, `twist_open`

slowest control cycle stayed at 11 ms against the 16.7 ms budget, with both cameras and the live view running.

7.2 Infrastructure fixes that mattered

Two failures would have silently corrupted a data session. In the virtualised setup, a leader's serial port could be opened once after plugging; every later open found no servo. Resetting the leaders' USB link before each open fixed it (3/3 consecutive sessions opened, against 1/2 without). The top-view camera delivers about 11 fps in the virtual machine whatever the format; the recorder stores the latest frame at 30 fps, so the dataset stays synchronous with the joints and the policy sees the same duplicated-frame pattern at inference. The temperature guard caught the joint 3 overheat of §4.2 before damage; with measured limits, no run has hit the guard since.

7.3 Operators get faster

Episodes got shorter as operators learned the task (Fig. 9): from 27.3 s on average over the first ten to 25.4 s over the last ten (median 26.6 s overall), with fewer hesitations—the long outliers are recovered grasps. Faster demonstrations are not only cheaper: consistent, decisive demonstrations are easier to imitate [30], which argues for a short warm-up session before recording a dataset.

7.4 Policy success: decomposition wins

Table 10 and Fig. 10 compare the policies trained on all 30 episodes. Chaining the four per-skill ACT policies completes the task in 17/20 trials, against 7/20 for one ACT trained on whole demonstrations and 14/20 for $\pi_{0.5}$ with egocentric co-training (11/20 without). Each skill alone is reliable (Table 11): locate and release never failed, grasp is the weakest link, and the chain's success is the product

Table 9: Simulated market: top bids per episode (EUR) with raw gap names and with names grouped by verb.

	Raw name	Bid	Grouped	Bid
1	`press_to_surface`	10.41	`press`	20.21
2	`tilt_out`	7.39	`drag`	14.45
3	`run_blender`	6.54	`tilt`	10.74
4	`drag_along_surface`	6.28	`twist`	10.40
5	`peel_item`	4.96	`peel`	9.94
6	`twist_item`	3.86	`run_blender`	8.79

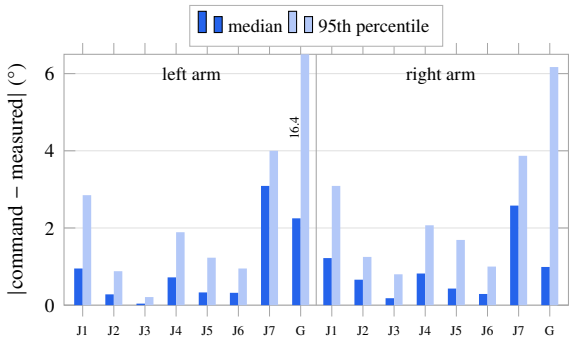


Figure 8: Tracking error per joint over the 30 demonstrations (24,412 frames): command sent to the follower vs its measured position. G: gripper (left 95th percentile clipped, 16.4°, while squeezing an object).

of its parts, so errors do not compound beyond it. With the same data, decomposition beats a pretrained VLA. The per-skill policies never have to infer the phase of the task from pixels; the whole-task ACT mostly fails by mixing phases, closing the gripper before reaching the tea bag or hesitating between objects. $\pi_{0.5}$ copes better with placement but drops the tea bag at insertion. Egocentric co-training still helps the VLA (+15 points).

7.5 Decomposition makes every episode count

Fig. 11 trains each policy on the first 10, 20 and 30 episodes. The horizontal reading gives two discounts. Egocentric video lets $\pi_{0.5}$ reach with 20 robot episodes what teleoperation alone reaches with 30, a third fewer, close to the discount $\delta = 0.4$ the bidder applies to E_0 when a gap matches an open egocentric dataset (§3.5). Decomposition is the larger lever: the per-skill chain reaches the same success with 10 episodes, a third of the data, and its skills are reused by every plan that binds them, so their data is paid for once.

7.6 The crop is worth a dataset

Training the per-skill policies on the uncropped top view and evaluating with the laptop moved and a person walking behind the table drops chained success to 45%; with the fixed crop it stays at 80% (85% with an undisturbed scene, Table 12). The distractors that a crop removes are exactly those that change between recording and deploy-

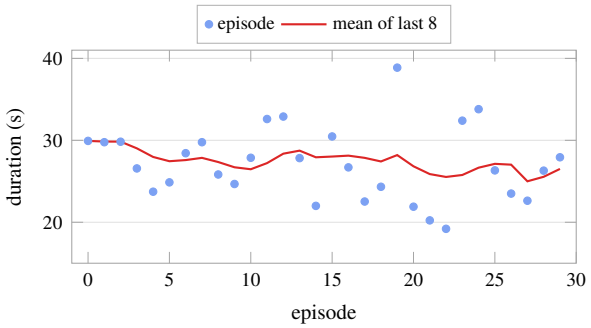


Figure 9: Duration of the 30 demonstrations in recording order.

Table 10: Grasp-and-insert success over 20 trials per policy (30 teleoperated episodes), with 95% Wilson intervals, and the main failure mode. [provisional data]

Policy	Data	Success	Main failure
ACT, whole task	teleop	7/20 [18,57]	phases mixed
$\pi_{0.5}$	teleop	11/20 [34,74]	dropped at insert
$\pi_{0.5}$	+ ego	14/20 [48,85]	dropped at insert
ACT per skill	teleop	**17/20** [64,95]	grasp slip

Table 11: Per-skill ACT policies, each evaluated alone from its own start state (20 trials), and chained. [provisional data]

Skill	Segments	Success
`locate_object`	30	20/20
`grasp_object`	30	18/20
`move_to_location`	30	19/20
`release_object`	30	20/20
Chained		17/20

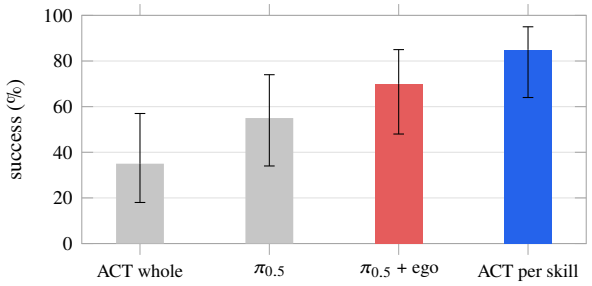


Figure 10: Grasp-and-insert success with 30 teleoperated episodes (20 trials each, 95% Wilson intervals). [provisional data]

ment, which is why we fixed the crop in the camera contract and re-encoded the old episodes instead of starting a new dataset.

7.7 From sentence to cup

End to end, the request "make me a cup of tea" compiles to a valid plan on every run; the first steps bind to the four newly trained skills and the pour to the scripted skill. Over 10 full runs with the per-skill ACT chain, 9 complete locate, grasp, move and release and the handover, and 8 pour within ±5 g of the requested mass. The handover itself never failed: torque stays on through the switch, so the held objects do not move (Table 13).

8 Discussion

A checker substitutes for scale. With two rounds of simulator feedback, planners roughly three times faster than the largest reach the same final validity. For an interactive store this is the right trade: the checker is cheap and exact, the model slow and approximate.

The data gap is the dexterity gap—and egocentric video narrows it. The motions compiled plans miss have almost no robot demonstrations and are matched to human video instead. Our robot experiments turn that observation into a lever: co-training on human video lets a VLA reach the same success with markedly fewer teleoperated episodes, so the store can price, and buy, the missing skills more cheaply than a robot-only data market would.

Decomposition beats pretraining at small data. With 30 episodes, four small ACT policies, one per atomic skill, outperform both a whole-task ACT and a fine-tuned $\pi_{0.5}$. The store's granularity is also the right learning granularity: the skills the compiler composes are short, single-phase behaviours that action chunking learns from a few dozen segments, they run without a GPU, and once trained they serve every plan that binds them. $\pi_{0.5}$ remains the stronger whole-task policy and gains most from egocentric video; we expect it to take over when a skill needs language or wider generalisation.

Safety is a property of the pipeline. Because the compiler can be wrong, installation and dispatch re-run the simulator rather than trusting a stored verdict, a queued plan cannot be edited, and an operator stop never sends the arm home. The same principle runs down to the motors: torque is enabled only after a read-only preflight and a typed `GO`, limits are measured rather than assumed, and a learned policy runs inside the exact loop that a human teleoperates through. None of these gates depend on the model, or the operator, behaving well.

Train and deploy on the same pixels. Declaring each camera once, with its rotation, crop and zoom, and reading that declaration in both the recorder and the runner removed a whole class of silent deployment failures. Changing the contract later is possible, but only by rewriting the dataset with it.

From valid to certified. A store should publish, per skill and per robot, a measured success rate with a confidence interval; with 20 trials, 18/20 is 90% with a Wilson interval of 70–97%. Validity is the necessary first gate; measured success is the product.

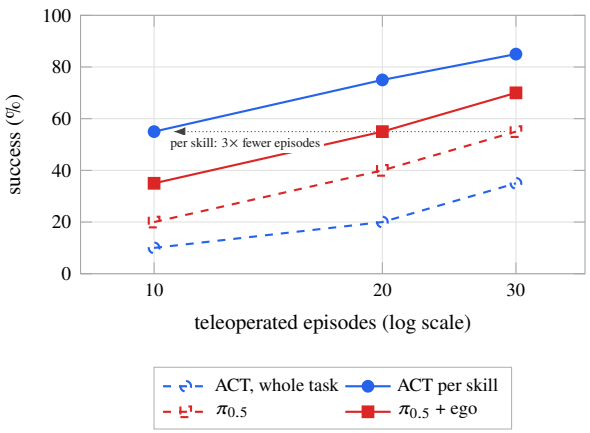


Figure 11: Data scaling: success against the number of teleoperated episodes (20 trials per point). [provisional data]

Table 12: Effect of the top-view crop on the chained per-skill ACT policies (20 trials). [provisional data]

Scene at test time	Uncropped	Cropped
As recorded	80%	85%
Laptop moved, person behind	45%	80%

9 Limitations

Valid is not successful. The simulator checks what the catalog declares; a plan it accepts can still fail on hardware. **The requests are not a held-out test set.** Prompts and start-state assumptions were revised while inspecting failures on these same requests. **Small samples.** Thirteen requests and 3 runs per planner, and 20 trials per policy, give wide intervals; differences of a few points are not significant. **One task, one robot.** Policy results cover one bimanual task on one OpenArm in one room; the egocentric discount may differ for motions whose human and robot embodiments diverge more (in-hand rotation, tool use with fingers). **Segmentation is semi-automatic.** Skill boundaries come from gripper events and were checked by hand. **Retargeting is approximate.** Human wrist poses are mapped to a 7-DoF arm with a parallel gripper; finger-level detail is lost. **The market is simulated.** Bids come from our own compilations; real demand will differ. **Scheduling ignores gaps' timing**, since gaps have no duration until they are trained.

10 Conclusion

Separating *acting out* a request from *binding* it to products, and letting a declarative simulator veto and explain, turns language models into usable compilers for a skill store. Closing the loop on a real bimanual robot turns the store's gaps into skills: a safety-gated teleoperation stack with measured limits and a frozen camera contract produces clean

Table 13: End-to-end runs of "make me a cup of tea" (per-skill ACT, scripted pour). [provisional data]

Stage	Succeeded
Plan compiled and valid	10/10
Learned skills, chained	9/10
Handover, torque kept on	9/9
Pour within ±5 g (scripted)	8/9
Full request	8/10

demonstrations, and splitting them into the catalog's atomic skills, one ACT policy each, yields a chain that beats a fine-tuned VLA trained on the same 30 episodes and hands over to a scripted pour. Human video is not only where the compiler finds candidate data for missing skills; it is what makes those skills cheap enough to buy. Next steps are canonicalising gap names into a motion taxonomy, training the most demanded gaps from egocentric data first, and publishing measured success rates per skill.

References

[1] M. Ahn, A. Brohan, N. Brown, et al. Do as I can, not as I say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.

[2] K. Black, N. Brown, D. Driess, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[3] A. Brohan, N. Brown, J. Carbajal, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *CoRL*, 2023.

[4] R. Cadene, S. Alibert, A. Soare, et al. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. https://github.com/huggingface/lerobot, 2024.

[5] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion Policy: Visuomotor policy learning via action diffusion. In *RSS*, 2023.

[6] D. Damen, H. Doughty, G. M. Farinella, et al. Rescaling egocentric vision: Collection, pipeline and challenges for EPIC-KITCHENS-100. *IJCV*, 130:33–55, 2022.

[7] Enactic. OpenArm: An open-source robot arm for physical AI research. https://github.com/enactic/openarm, 2025.

[8] R. E. Fikes and N. J. Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971.

[9] Z. Fu, T. Z. Zhao, and C. Finn. Mobile ALOHA: Learning bimanual mobile manipulation with low-cost whole-body teleoperation. In *CoRL*, 2024.

[10] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez. Integrated task and motion planning. *Annual Review of Control, Robotics, and Autonomous Systems*, 4:265–293, 2021.

[11] K. Grauman, A. Westbury, E. Byrne, et al. Ego4D: Around the world in 3,000 hours of egocentric video. In *CVPR*, 2022.

[12] K. Grauman, A. Westbury, L. Torresani, et al. Ego-Exo4D:

Understanding skilled human activity from first- and third-person perspectives. In *CVPR*, 2024.

[13] N. Hogan. Impedance control: An approach to manipulation. *ASME Journal of Dynamic Systems, Measurement, and Control*, 107(1):1–24, 1985.

[14] R. Hoque, P. Huang, D. J. Yoon, M. Sivapurapu, and J. Zhang. EgoDex: Learning dexterous manipulation from large-scale egocentric video. *arXiv preprint arXiv:2505.11709*, 2025.

[15] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *ICML*, 2022.

[16] W. Huang, F. Xia, T. Xiao, et al. Inner monologue: Embodied reasoning through planning with language models. In *CoRL*, 2022.

[17] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei. VoxPoser: Composable 3D value maps for robotic manipulation with language models. In *CoRL*, 2023.

[18] ISO/TS 15066:2016. Robots and robotic devices—Collaborative robots. International Organization for Standardization, 2016.

[19] S. Kambhampati, K. Valmeekam, L. Guan, et al. Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks. In *ICML*, 2024.

[20] S. Kareer, D. Patel, R. Punamiya, P. Mathur, S. Cheng, C. Wang, J. Hoffman, and D. Xu. EgoMimic: Scaling imitation learning via egocentric video. *arXiv preprint arXiv:2410.24221*, 2024.

[21] A. Khazatsky, K. Pertsch, S. Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *RSS*, 2024.

[22] M. J. Kim, K. Pertsch, S. Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *CoRL*, 2024.

[23] G. Konidaris and A. Barto. Skill discovery in continuous reinforcement learning domains using skill chaining. In *NeurIPS*, 2009.

[24] I. Kostrikov, D. Yarats, and R. Fergus. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In *ICLR*, 2021.

[25] M. Laskin, K. Lee, A. Stooke, L. Pinto, P. Abbeel, and A. Srinivas. Reinforcement learning with augmented data. In *NeurIPS*, 2020.

[26] J. Liang, W. Huang, F. Xia, et al. Code as policies: Language model programs for embodied control. In *ICRA*, 2023.

[27] K. Lin, C. Agia, T. Migimatsu, M. Pavone, and J. Bohg. Text2Motion: From natural language instructions to feasible plans. *Autonomous Robots*, 47:1345–1365, 2023.

[28] F. Lin, Y. Hu, P. Sheng, C. Wen, J. You, and Y. Gao. Data scaling laws in imitation learning for robotic manipulation. In *ICLR*, 2025.

[29] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone. LLM+P: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.

[30] A. Mandlekar, D. Xu, J. Wong, et al. What matters in learning from offline human demonstrations for robot manipulation. In *CoRL*, 2021.

[31] D. McDermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld, and D. Wilkins. PDDL—the planning domain definition language. Technical Report CVC TR-98-003, Yale Center for Computational Vision and Control, 1998.

[32] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta. R3M: A universal visual representation for robot manipulation. In *CoRL*, 2022.

[33] NVIDIA, J. Bjorck, F. Castañeda, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

[34] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, et al. Octo: An open-source generalist robot policy. In *RSS*, 2024.

[35] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and RT-X models. In *ICRA*, 2024.

[36] Physical Intelligence, K. Black, N. Brown, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[37] R.-Z. Qiu, S. Yang, X. Cheng, et al. Humanoid policy ~ human policy. *arXiv preprint arXiv:2503.13441*, 2025.

[38] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo. *Robotics: Modelling, Planning and Control*. Springer, 2009.

[39] I. Singh, V. Blukis, A. Mousavian, et al. ProgPrompt: Generating situated robot task plans using large language models. In *ICRA*, 2023.

[40] R. S. Sutton, D. Precup, and S. Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.

[41] K. Valmeekam, M. Marquez, A. Olmo, S. Sreedharan, and S. Kambhampati. PlanBench: An extensible benchmark for evaluating large language models on planning and reasoning about change. In *NeurIPS Datasets and Benchmarks*, 2023.

[42] C. Wang, L. Fan, J. Sun, R. Zhang, L. Fei-Fei, D. Xu, Y. Zhu, and A. Anandkumar. MimicPlay: Long-horizon imitation learning by watching human play. In *CoRL*, 2023.

[43] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *JASA*, 22(158):209–212, 1927.

[44] P. Wu, Y. Shentu, Z. Yi, X. Lin, and P. Abbeel. GELLO: A general, low-cost, and intuitive teleoperation framework for robot manipulators. In *IROS*, 2024.

[45] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *RSS*, 2023.

[46] T. Z. Zhao, J. Tompson, D. Driess, et al. ALOHA Unleashed: A simple recipe for robot dexterity. In *CoRL*, 2024.

A Requests and object kinds

Unfamiliar requests (N). "You have a paint bucket and brush. draw an image on the table"; "build a small sandcastle on the tray with the bucket and shovel"; "do a simple card trick with the deck on the table"; "water the two plants on the shelf"; "write my name on the whiteboard with the marker"; "make me a jam sandwich"; "put the flowers in the vase and add water"; "feed the cat: open the tin and put the food in her bowl". **Demo requests (D).** "make me a pina colada"; "pour me a glass of juice"; "mix me a smoothie"; "wipe my table clean after dinner"; "clear the table and put

the bottles away".

Object kinds. `bottle`: a rigid bottle, carton or jug of liquid, held upright; `glass`: a drinking glass, cup or mug; `shaker`: a cocktail shaker; `scoop`: a measuring scoop with a handle; `cloth`: something soft and deformable; `item`: any other rigid, hand-sized loose thing; `fixed`: something that is not picked up (a table, a shelf, an appliance); `substance`: a material inside something else (paint, water, sand).

B Simulator checks

Table 14: Every check the simulator runs; *adv.*: advisory, returned to the binder but not blocking installation.

Check	Raised when
Unmet precondition	a step needs a fact, on its hand, that no earlier step or gap established
Wrong object kind	the skill's `acts_on` does not include the kind of its target
Wrong pour direction	the held object is the target, or is an empty glass or shaker
Embodiment mismatch	the skill needs a body (torso, base) the robot lacks
Unknown skill	a step names a skill outside the catalog (prevented by the enum)
Empty plan	the binding contains no executable step
Ends holding (adv.)	a hand still holds or has located something at the end
Pointless move (adv.)	an object is picked up and set back where it was, unused

Repair hints. Each issue returns with a catalog-derived hint: the skills that establish a missing fact, the containers holding something, or the skill a duplicate gap restates.

C Robot configuration

Table 15: Robot, recording and training parameters.

Parameter	Value
Arms	2 × OpenArm v10, 7 DoF + gripper
Motors	Damiao DM8009 / DM4340 / DM4310
Bus	2 × CAN-FD, 1 / 5 Mbps, sample point 0.75
Leaders	2 × OpenArm Mini (Feetech)
Control / dataset rate	60 Hz / 30 fps
Cameras	D455 (front), USB top view (back), 640×480
Camera preprocessing	front: rot. 180°, zoom 1.1×; back: crop 512×384 @ (16,40)
State / action	16-D joint positions (deg) / 16-D commands
Dataset	30 episodes, 24,412 frames, H.264, LeRobot v3
ACT	one per skill (4), chunk 100, ResNet-18, 50k steps, batch 8
$\pi_{0.5}$	base checkpoint, 30k steps, batch 32
Egocentric mix	25% of each batch, robot-only normalisation